Instances of `std::shared_future` that reference some asynchronous state are constructed from instances of `std::future` that reference that state. Since `std::future` objects don't share ownership of the asynchronous state with any other object, the ownership must be transferred into the `std::shared_future` using `std::move`, leaving the `std::future` in an empty state, as if it was default constructed:

```
std::promise<int> p;
std::future<int> f(p.get_future());
assert(f.valid());
std::shared_future<int> sf(std::move(f));
assert(!f.valid());
assert(sf.valid());
```

❶ The future f is valid

❷ f is no longer valid

❸ sf is now valid

Here, the future `f` is initially valid ❶ because it refers to the asynchronous state of the promise `p`, but after transferring the state to `sf, f` is no longer valid ❷, whereas `sf` is ❸. Just as with other movable objects, the transfer of ownership is implicit for rvalues, so you can construct a `std::shared_future` directly from the return value of the `get_future()` member function of a `std::promise` object, for example:

```
std::promise<std::string> p;
std::shared_future<std::string> sf(p.get_future());
```

❶ Implicit transfer of ownership

Here, the transfer of ownership is implicit; the `std::shared_future<>` is constructed from an rvalue of type `std::future<std::string>` ❶.

`std::future` also has an additional feature to facilitate the use of `std::shared_future` with the new facility for automatically deducing the type of a variable from its initializer (see appendix A, section A.6). `std::future` has a `share()` member function that creates a new `std::shared_future` and transfers ownership to it directly. This can save a lot of typing and makes code easier to change:

```
std::promise< std::map< SomeIndexType, SomeDataType, SomeComparator,
    SomeAllocator>::iterator> p;
auto sf=p.get_future().share();
```

In this case, the type of `sf` is deduced to be `std::shared_future< std::map< SomeIndexType, SomeDataType, SomeComparator, SomeAllocator>::iterator>`, which is rather a mouthful. If the comparator or allocator is changed, you only need to change the type of the promise; the type of the future is automatically updated to match.

Sometimes you want to limit the amount of time you're waiting for an event, either because you have a hard time limit on how long a particular section of code may take, or because there's other useful work that the thread can be doing if the event isn't going to happen soon. To handle this facility, many of the waiting functions have variants that allow a timeout to be specified.

4.3. Waiting with a time limit

All the blocking calls introduced previously will block for an indefinite period of time, suspending the thread until the event being waited for occurs. In many cases this is fine, but in some cases you